UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS3014 - Estructuras de Datos Avanzadas

Jerarquía de Memoria III
Semana 10

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

16 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión



- Construir el layout de van Emde Boas de un árbol binario de búsqueda

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir el layout de van Emde Boas de un árbol binario de búsqueda
- Demostrar que una búsqueda sobre ese layout cuesta $O(\log_B N)$ transferencias para cualquier B

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir el layout de van Emde Boas de un árbol binario de búsqueda
- Demostrar que una búsqueda sobre ese layout cuesta $O(\log_B N)$ transferencias para cualquier B
- Volver dinámica la estructura combinándola con un arreglo con huecos (Ordered File Maintenance)

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir el layout de van Emde Boas de un árbol binario de búsqueda
- Demostrar que una búsqueda sobre ese layout cuesta $O(\log_B N)$ transferencias para cualquier B
- Volver dinámica la estructura combinándola con un arreglo con huecos (Ordered File Maintenance)
- Usar indirección para que insertar y eliminar también cuesten $O(\log_B N)$

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dónde quedamos

En el modelo cache-oblivious el algoritmo no conoce B. Un árbol binario guardado por niveles, en orden o en preorden cuesta $\Theta(\lg(N/B))$ por búsqueda, lejos del $\Theta(\log_{B+1} N)$ del B-Tree.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dónde quedamos

En el modelo cache-oblivious el algoritmo no conoce B. Un árbol binario guardado por niveles, en orden o en preorden cuesta $\Theta(\lg(N/B))$ por búsqueda, lejos del $\Theta(\log_{B+1} N)$ del B-Tree.

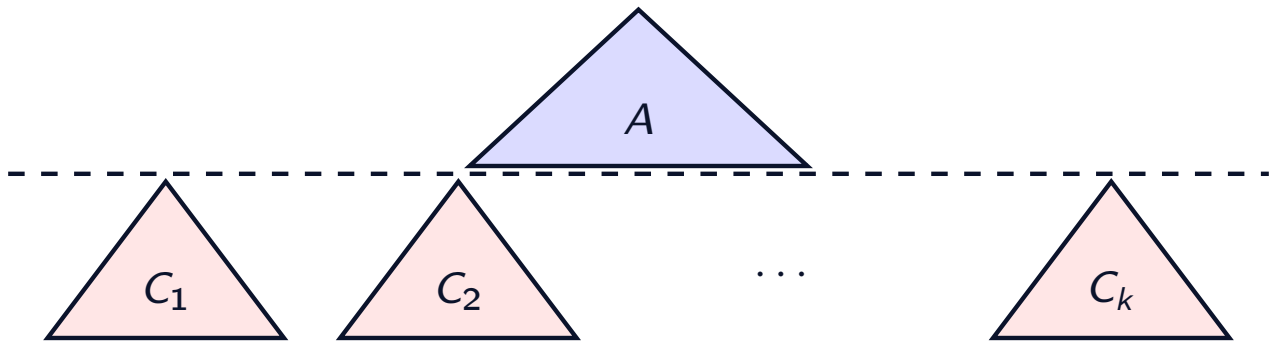
¿Se puede igualar al B-Tree sin saber cuánto vale B?

Hoy: sí, y en dos tiempos

Primero una estructura estática, que solo busca. Después la volvemos dinámica en cuatro pasos, apoyados en una caja negra que abriremos la próxima clase.

El layout de van Emde Boas

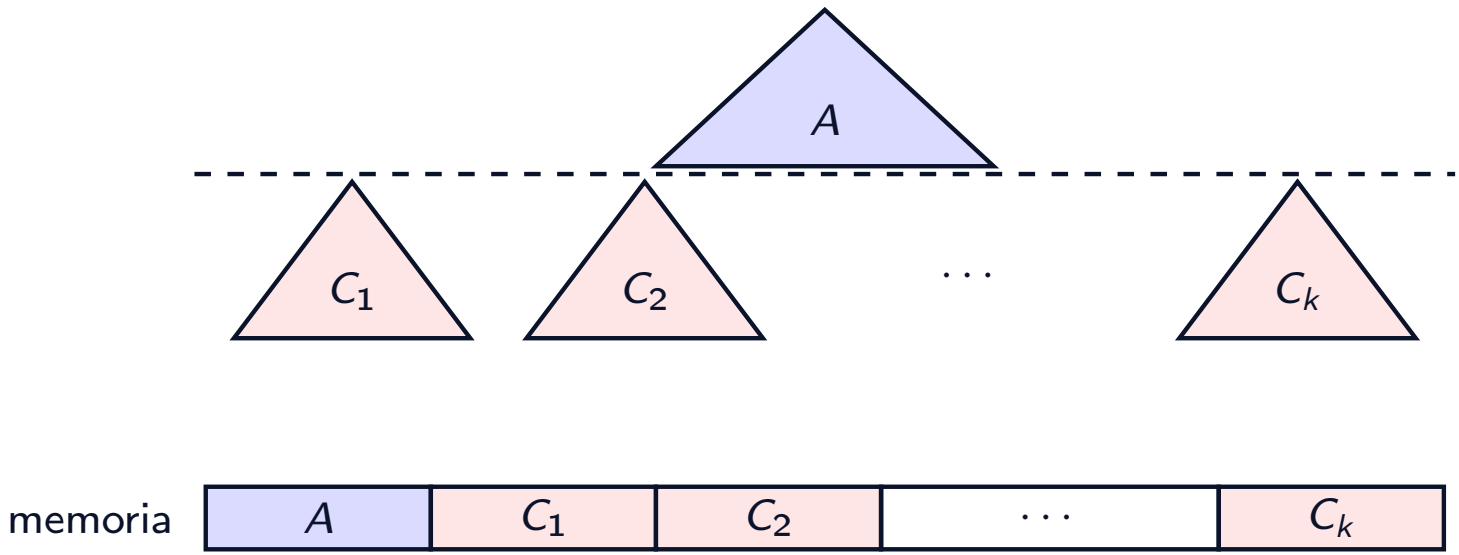
El layout de van Emde Boas



Definición recursiva

Se corta el árbol, de altura h, a la mitad de su altura: queda un árbol superior A de altura $h/2$ y, debajo, unos $\sqrt{N}$ subárboles $C_1, \ldots, C_k$ de altura $h/2$.

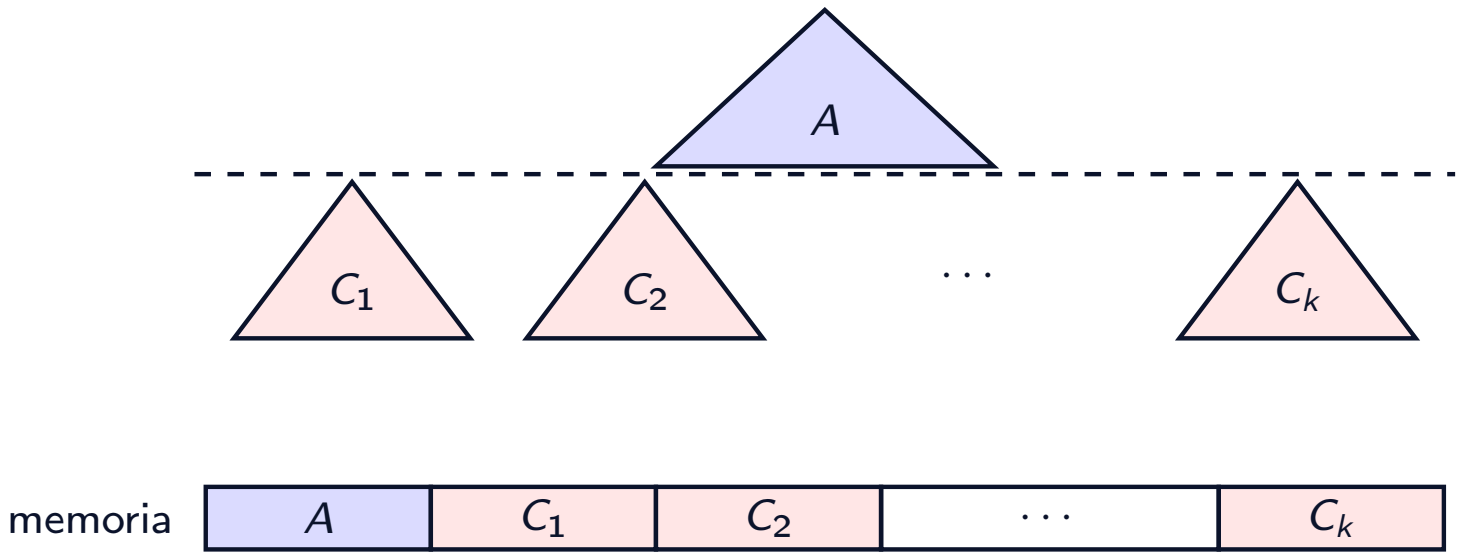
El layout de van Emde Boas



Definición recursiva

En memoria se guarda primero A y después $C_1, C_2, \ldots, C_k$, uno tras otro.

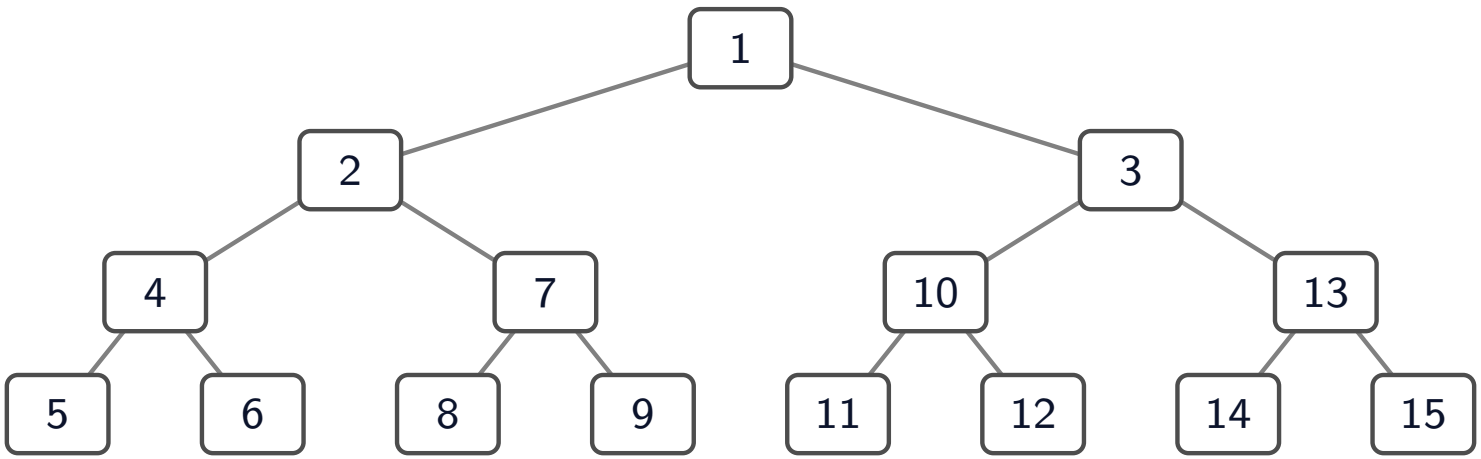
El layout de van Emde Boas



Definición recursiva

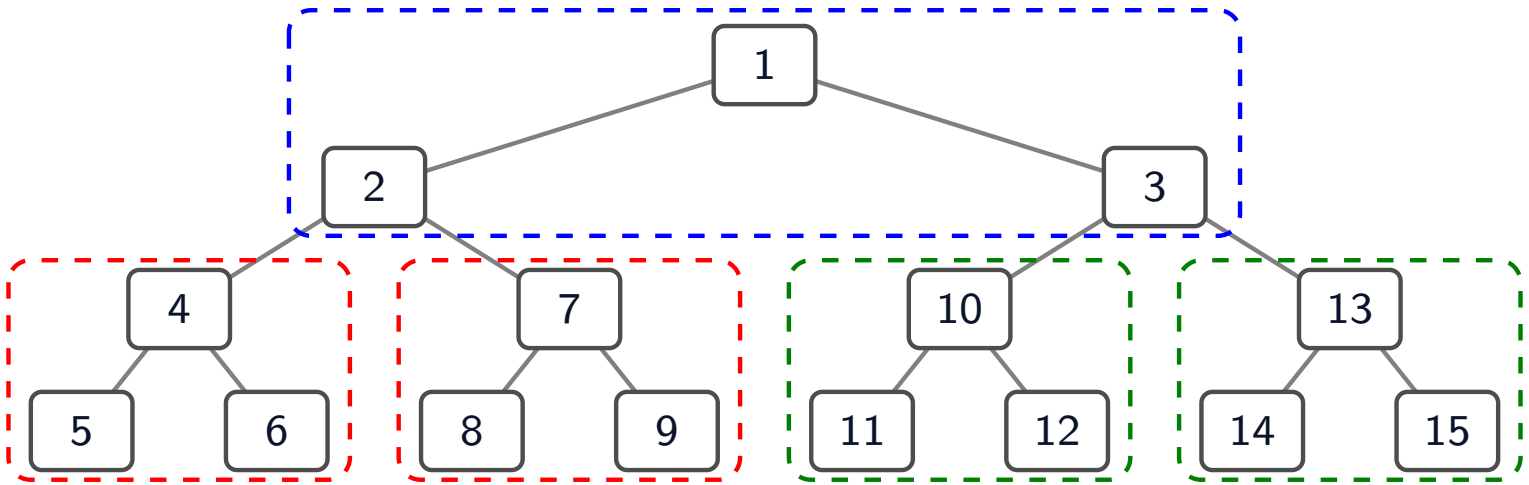
Y cada pieza se guarda, a su vez, con el mismo criterio: se corta a media altura y se repite, hasta llegar a nodos sueltos.

En un árbol de 15 nodos



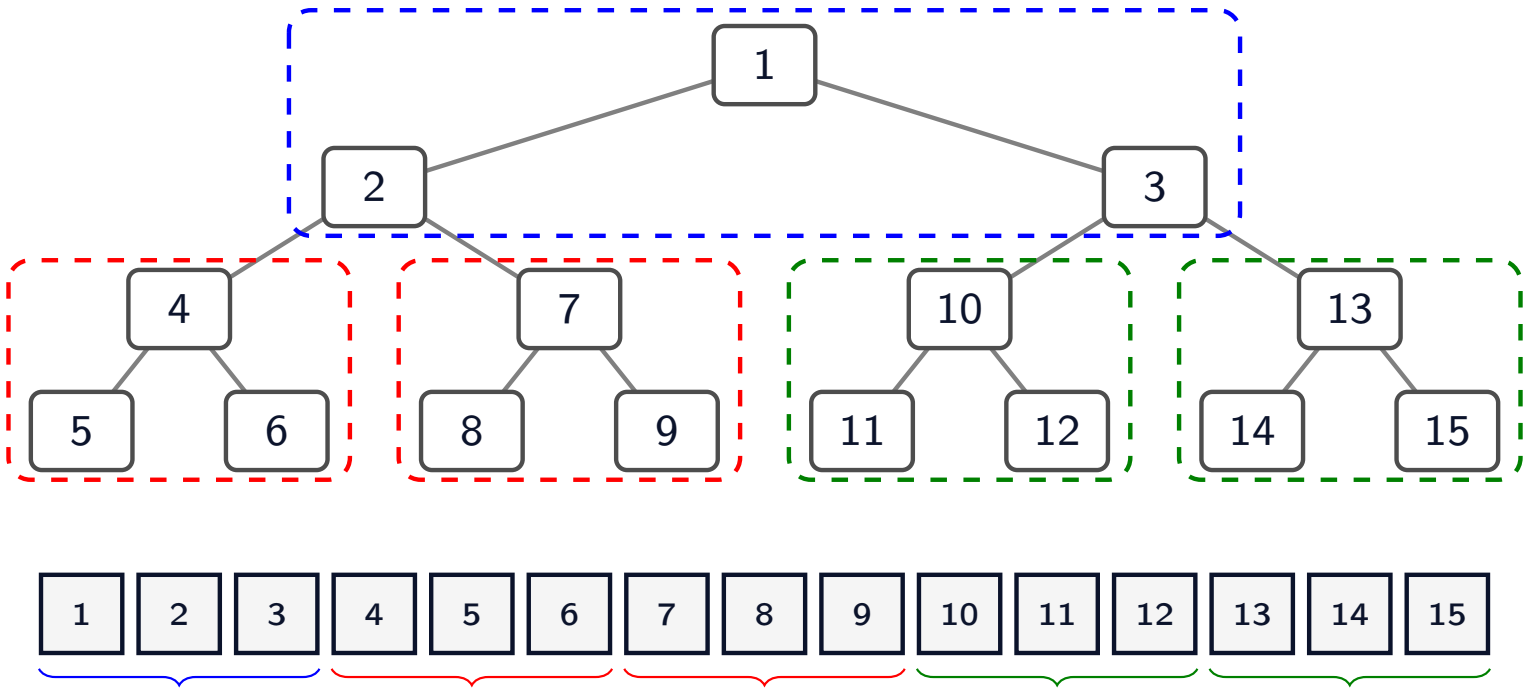
Altura 4. Cada nodo muestra la posición de memoria que le toca.

En un árbol de 15 nodos



El corte a media altura deja una pieza superior de 3 nodos y cuatro piezas inferiores de 3 nodos.

En un árbol de 15 nodos



En memoria: la pieza superior y luego las cuatro inferiores, cada una en celdas consecutivas.

¿En qué se diferencia del orden por niveles?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Layout	Orden en que se guardan los nodos
Por niveles	$1, 2, 3,\ \ 4, 5, 6, 7,\ \ 8, 9, 10, 11, 12, 13, 14, 15$
van Emde Boas	$1, 2, 3,\ \ \ 4, 8, 9,\ \ \ 5, 10, 11,\ \ \ 6, 12, 13,\ \ \ 7, 14, 15$

¿En qué se diferencia del orden por niveles?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Layout	Orden en que se guardan los nodos
Por niveles	$1, 2, 3,\ 4, 5, 6, 7,\ 8, 9, 10, 11, 12, 13, 14, 15$
van Emde Boas	$1, 2, 3,\ 4, 8, 9,\ 5, 10, 11,\ 6, 12, 13,\ 7, 14, 15$

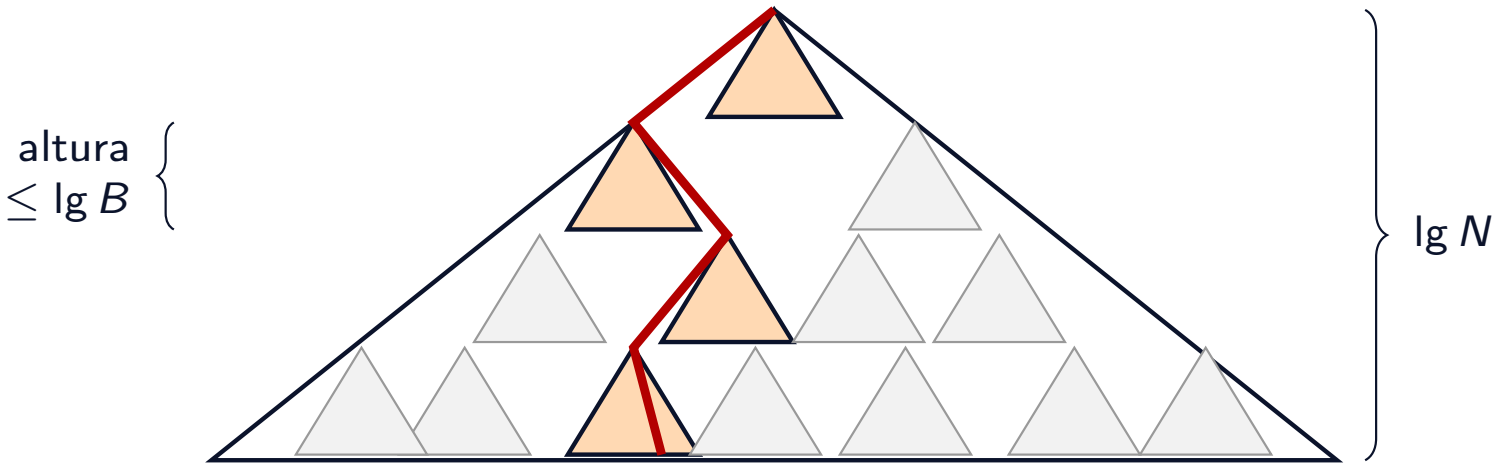
Un nodo queda junto a sus hijos, en todas las escalas

Numerando los nodos por niveles: en ese orden, el nodo 4 está a cuatro celdas de sus hijos 8 y 9, y esa distancia se duplica en cada nivel. En el layout de van Emde Boas, 4, 8 y 9 son vecinos.

Y el layout no menciona a B

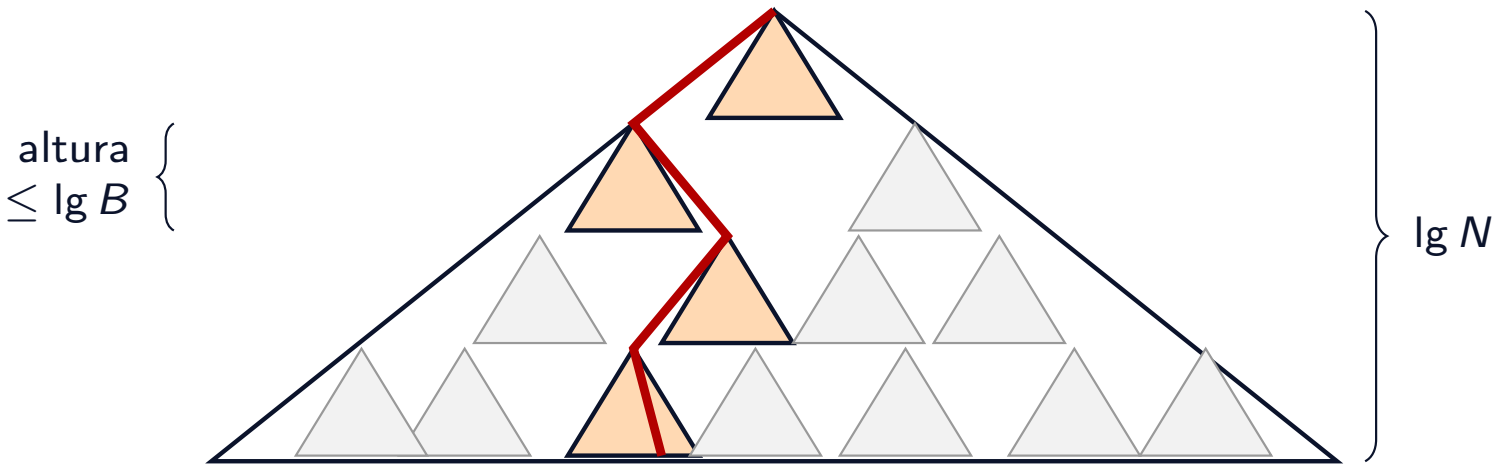
La recursión baja hasta nodos sueltos. No hay ningún parámetro que elegir: es un orden fijo de los nodos del árbol.

¿Cuánto cuesta una búsqueda?



El análisis sí puede usar B

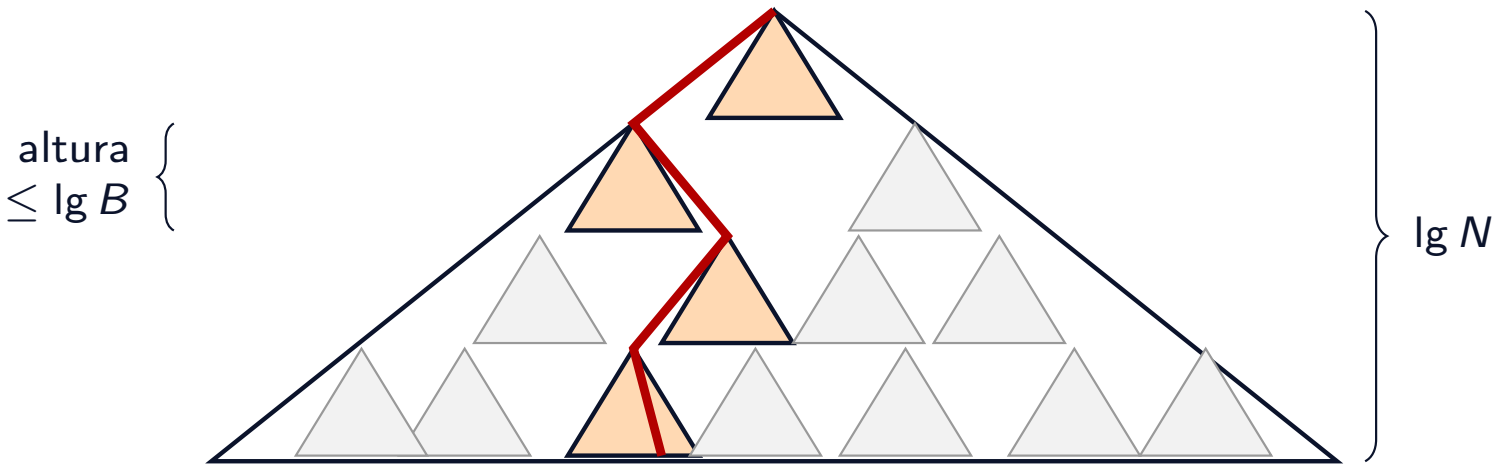
¿Cuánto cuesta una búsqueda?



El análisis sí puede usar B

Seguimos la recursión del layout hasta el primer nivel en que las piezas tienen altura $\leq \lg B$. En ese nivel, cada pieza tiene altura entre $\frac{1}{2} \lg B$ y $\lg B$: entre $\sqrt{B}$ y B nodos.

¿Cuánto cuesta una búsqueda?



El análisis sí puede usar B

Seguimos la recursión del layout hasta el primer nivel en que las piezas tienen altura $\leq \lg B$. En ese nivel, cada pieza tiene altura entre $\frac{1}{2} \lg B$ y $\lg B$: entre $\sqrt{B}$ y B nodos.

Una búsqueda es un camino de la raíz a una hoja: atraviesa algunas de esas piezas, una debajo de otra.

Contando piezas y bloques

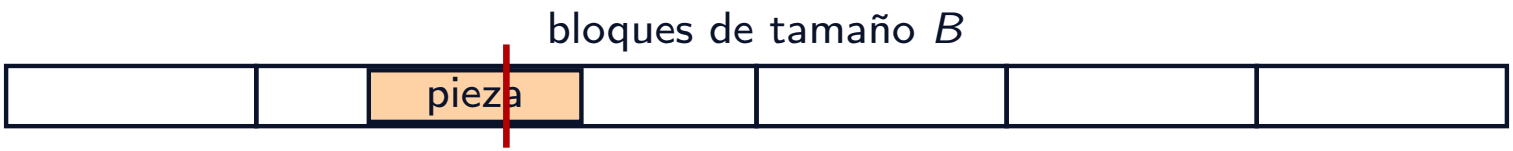
¿Cuántas piezas atraviesa el camino?

El camino mide $\lg N$ y cada pieza aporta al menos $\frac{1}{2} \lg B$: a lo más $2 \log_B N$ piezas.

Contando piezas y bloques

¿Cuántas piezas atraviesa el camino?

El camino mide $\lg N$ y cada pieza aporta al menos $\frac{1}{2} \lg B$: a lo más $2 \log_B N$ piezas.



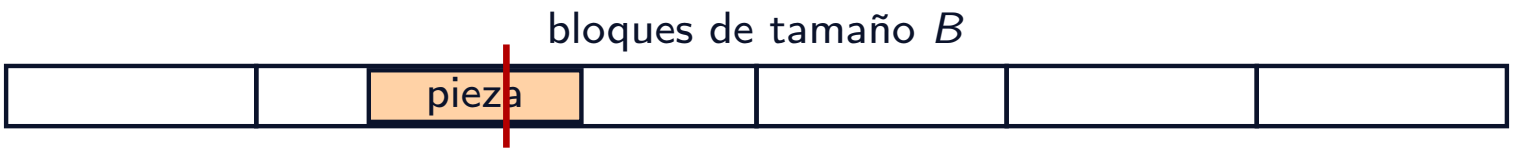
¿Cuántos bloques ocupa una pieza?

Tiene a lo más B nodos **consecutivos**: cabe en un bloque, salvo que caiga partida entre dos. A lo más 2.

Contando piezas y bloques

¿Cuántas piezas atraviesa el camino?

El camino mide $\lg N$ y cada pieza aporta al menos $\frac{1}{2} \lg B$: a lo más $2 \log_B N$ piezas.



¿Cuántos bloques ocupa una pieza?

Tiene a lo más B nodos **consecutivos**: cabe en un bloque, salvo que caiga partida entre dos. A lo más 2.

Teorema

Buscar en un árbol con layout de van Emde Boas cuesta a lo más $4 \log_B N = O(\log_{B+1} N)$ transferencias, para todo B.

El tablero de layouts

Estructura	Transferencias	¿Conoce B?
Nodos dispersos	$\Theta(\lg N)$	No
Por niveles, en orden, preorden	$\Theta(\lg(N/B))$	No
B-Tree	$\Theta(\log_{B+1} N)$	Sí
Layout de van Emde Boas	$O(\log_{B+1} N)$	No

Óptimo en todas las fronteras de la jerarquía a la vez

El mismo arreglo es eficiente entre caché y RAM y entre RAM y disco: el análisis vale para cada B por separado.

El tablero de layouts

Estructura	Transferencias	¿Conoce B?
Nodos dispersos	$\Theta(\lg N)$	No
Por niveles, en orden, preorden	$\Theta(\lg(N/B))$	No
B-Tree	$\Theta(\log_{B+1} N)$	Sí
Layout de van Emde Boas	$O(\log_{B+1} N)$	No

Óptimo en todas las fronteras de la jerarquía a la vez

El mismo arreglo es eficiente entre caché y RAM y entre RAM y disco: el análisis vale para cada B por separado.

Pero es una estructura **estática**

El árbol es perfecto y sus nodos tienen una posición fija en un arreglo. No sabemos insertar ni eliminar.

Volviéndonos dinámicos

¿Qué se rompe al insertar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos problemas distintos

- El arreglo no tiene lugar: meter una clave en medio obliga a desplazar todo lo que viene después.
- La forma del árbol es rígida: es un árbol perfecto, y su layout depende de esa forma.

¿Qué se rompe al insertar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

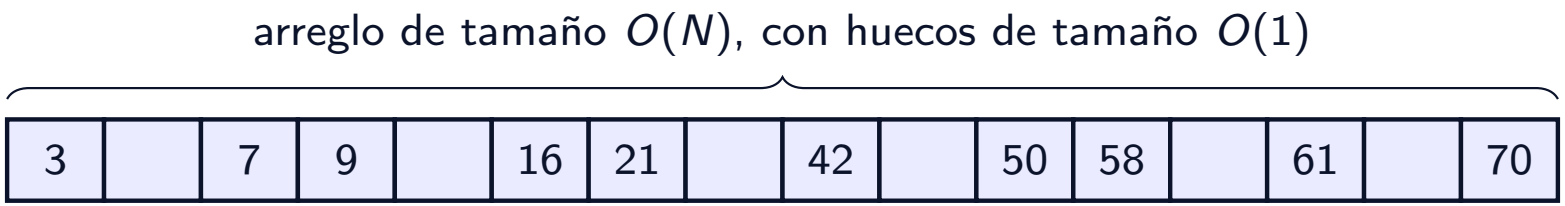
Dos problemas distintos

- El arreglo no tiene lugar: meter una clave en medio obliga a desplazar todo lo que viene después.
- La forma del árbol es rígida: es un árbol perfecto, y su layout depende de esa forma.

La estrategia

Separar las dos cosas. Las claves vivirán en un arreglo **con huecos**, pensado para insertar. El árbol con layout de van Emde Boas quedará encima, como índice, y su forma casi nunca cambiará.

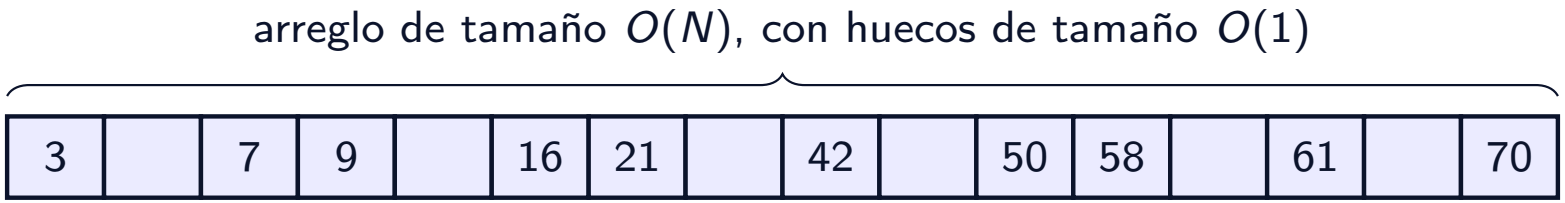
Ordered File Maintenance (OFM)



El problema

- Guardar N elementos, en un orden dado, en un arreglo de tamaño $O(N)$.
- Entre dos elementos consecutivos hay a lo más $O(1)$ celdas vacías: recorrer k elementos sigue costando $O(1 + k/B)$.

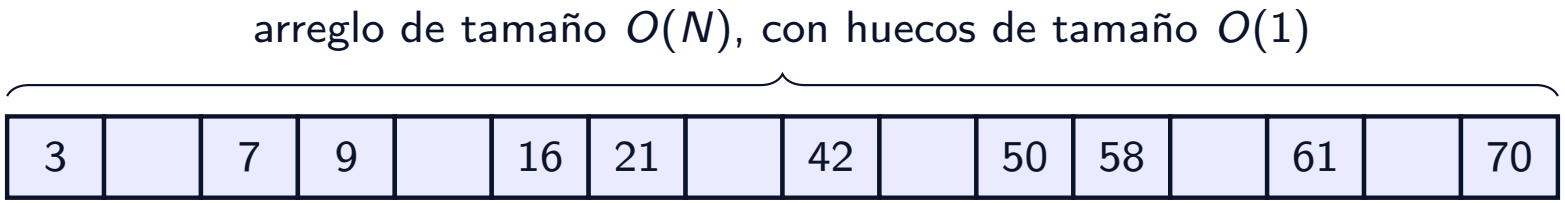
Ordered File Maintenance (OFM)



El problema

- Guardar N elementos, en un orden dado, en un arreglo de tamaño $O(N)$.
- Entre dos elementos consecutivos hay a lo más $O(1)$ celdas vacías: recorrer k elementos sigue costando $O(1 + k/B)$.
- INSERT y DELETE reacomodan $O(\lg^2 N)$ celdas **consecutivas**, amortizado: $O\left(1 + \frac{\lg^2 N}{B}\right)$ transferencias.

Ordered File Maintenance (OFM)

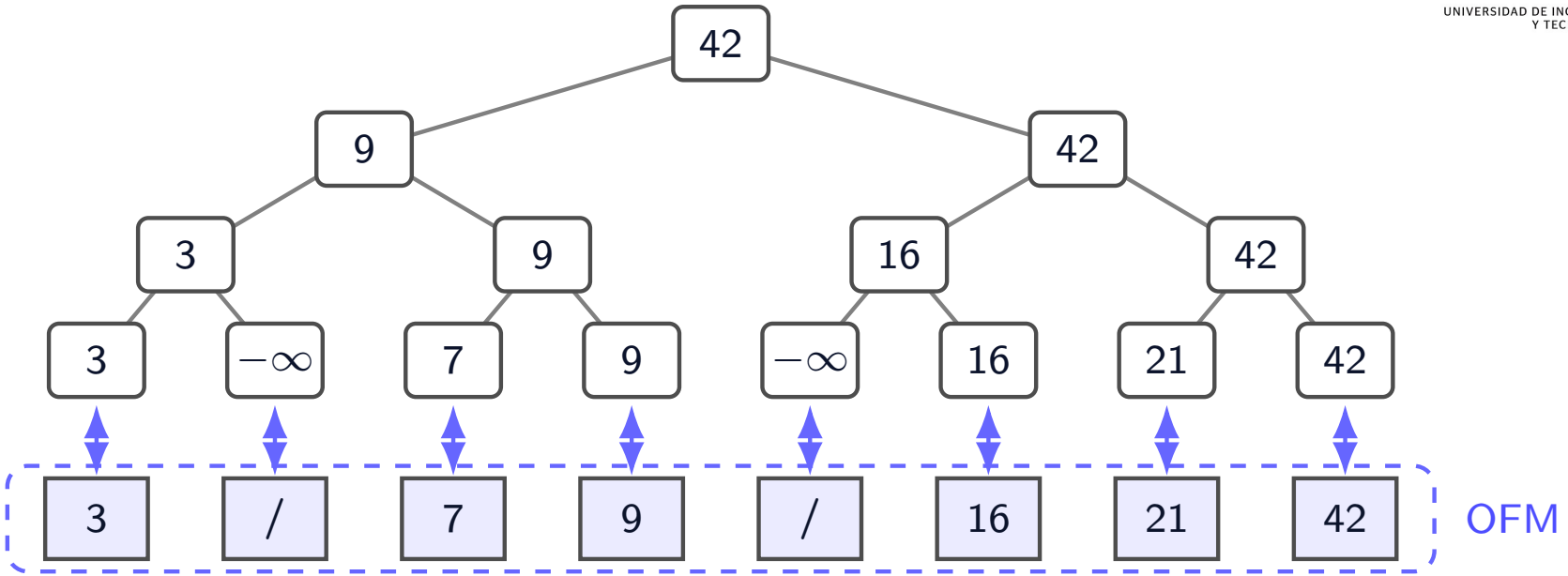


El problema

- Guardar N elementos, en un orden dado, en un arreglo de tamaño $O(N)$.
- Entre dos elementos consecutivos hay a lo más $O(1)$ celdas vacías: recorrer k elementos sigue costando $O(1 + k/B)$.
- INSERT y DELETE reacomodan $O(\lg^2 N)$ celdas **consecutivas**, amortizado: $O\left(1 + \frac{\lg^2 N}{B}\right)$ transferencias.

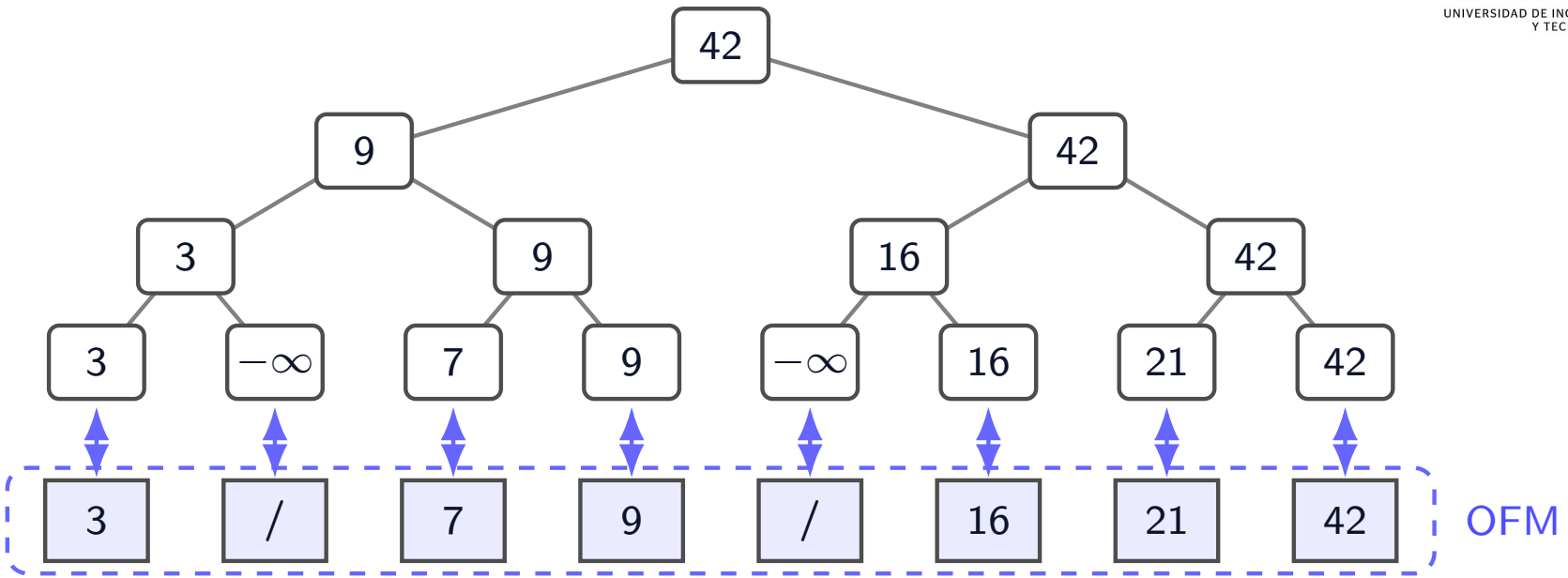
Por ahora es una caja negra. Cómo se logra es el tema de la próxima clase.

Paso 1: un árbol sobre el OFM



La construcción

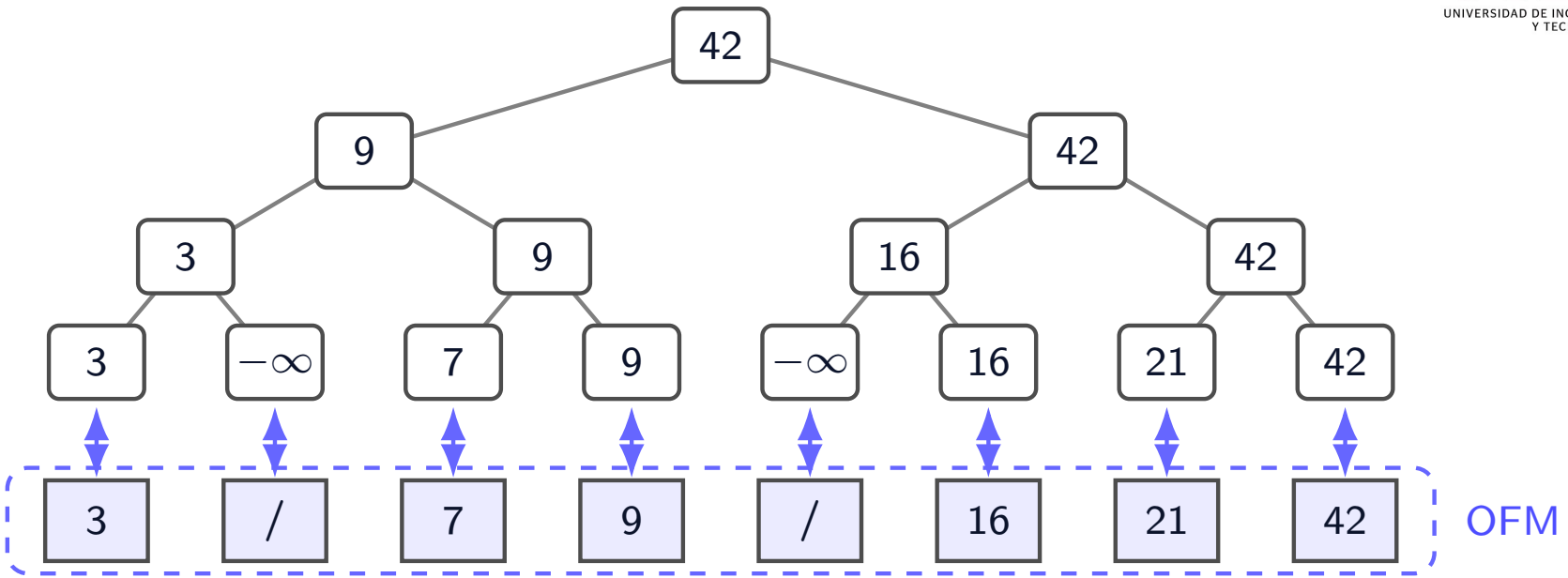
Paso 1: un árbol sobre el OFM



La construcción

Las celdas del OFM son las **hojas** de un árbol binario perfecto, guardado con layout de van Emde Boas. Cada hoja está enlazada con su celda, en ambos sentidos.

Paso 1: un árbol sobre el OFM

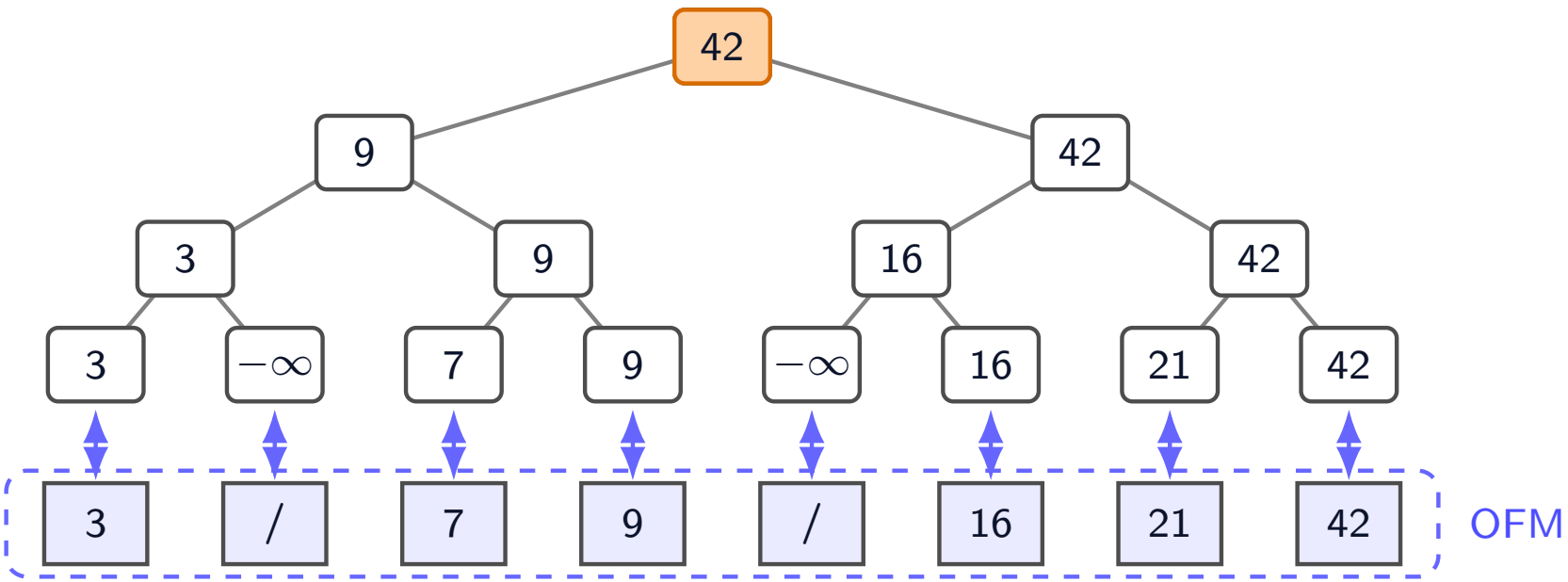


La construcción

Las celdas del OFM son las **hojas** de un árbol binario perfecto, guardado con layout de van Emde Boas. Cada hoja está enlazada con su celda, en ambos sentidos.

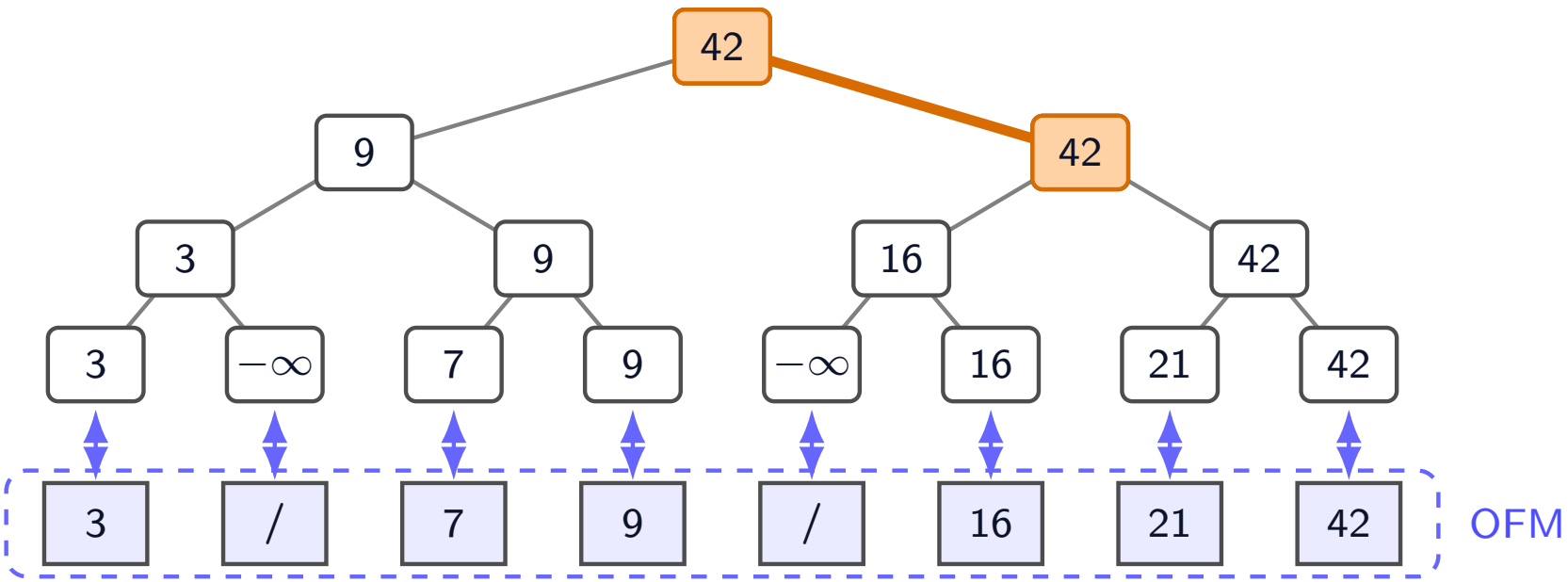
Cada nodo interno guarda el **máximo** de su subárbol. Un hueco ("/") cuenta como $-\infty$.

Buscar



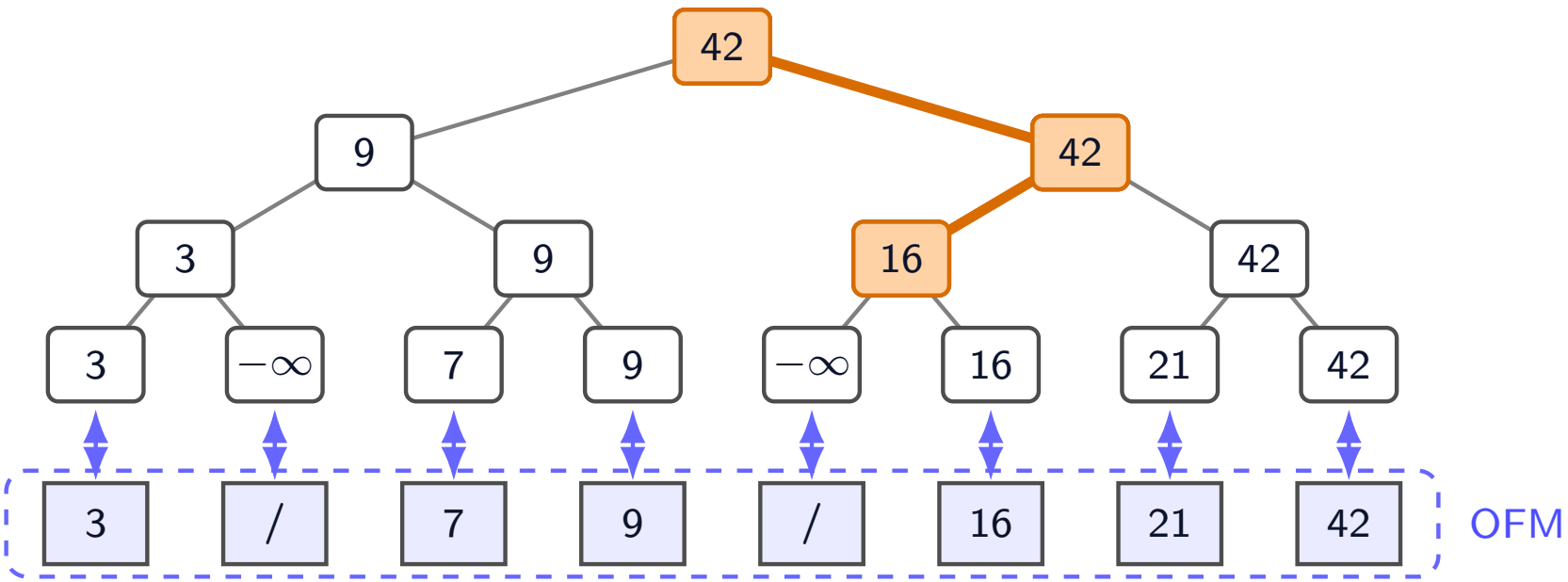
Buscar el 16. En cada nodo miramos el máximo de su hijo izquierdo.

Buscar



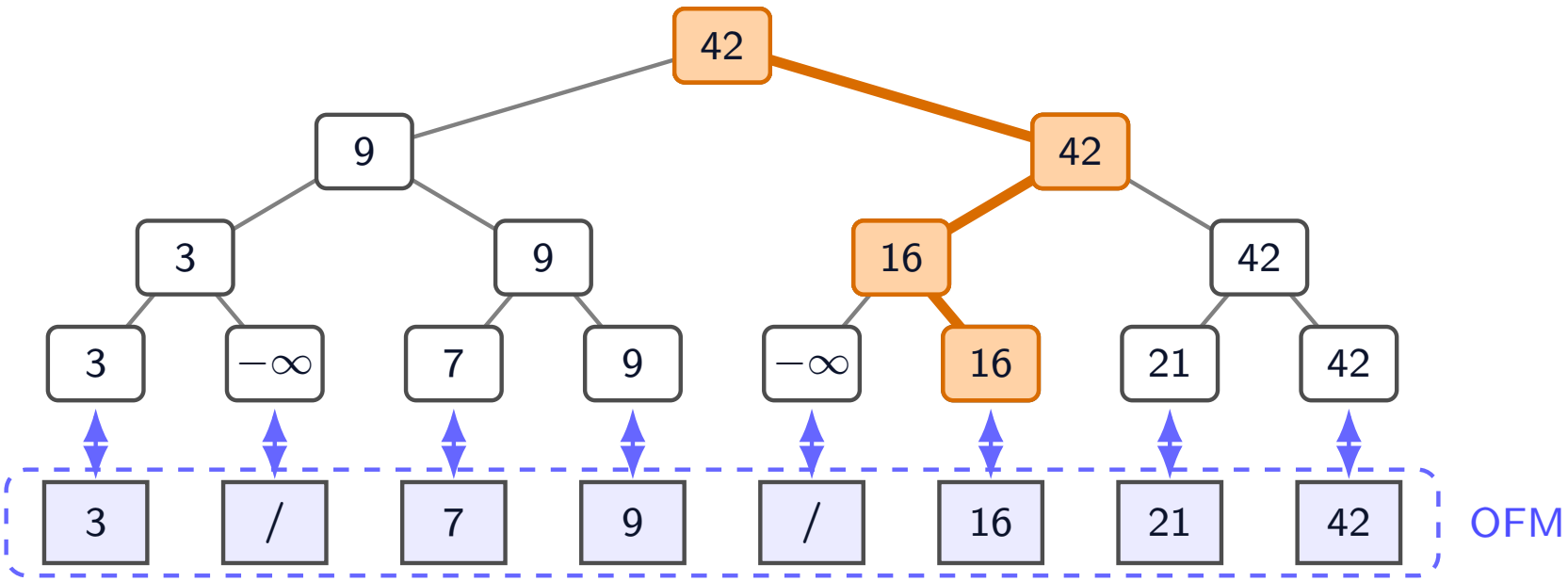
El hijo izquierdo de la raíz tiene máximo $9 < 16$: el 16 no está ahí. Vamos a la derecha.

Buscar



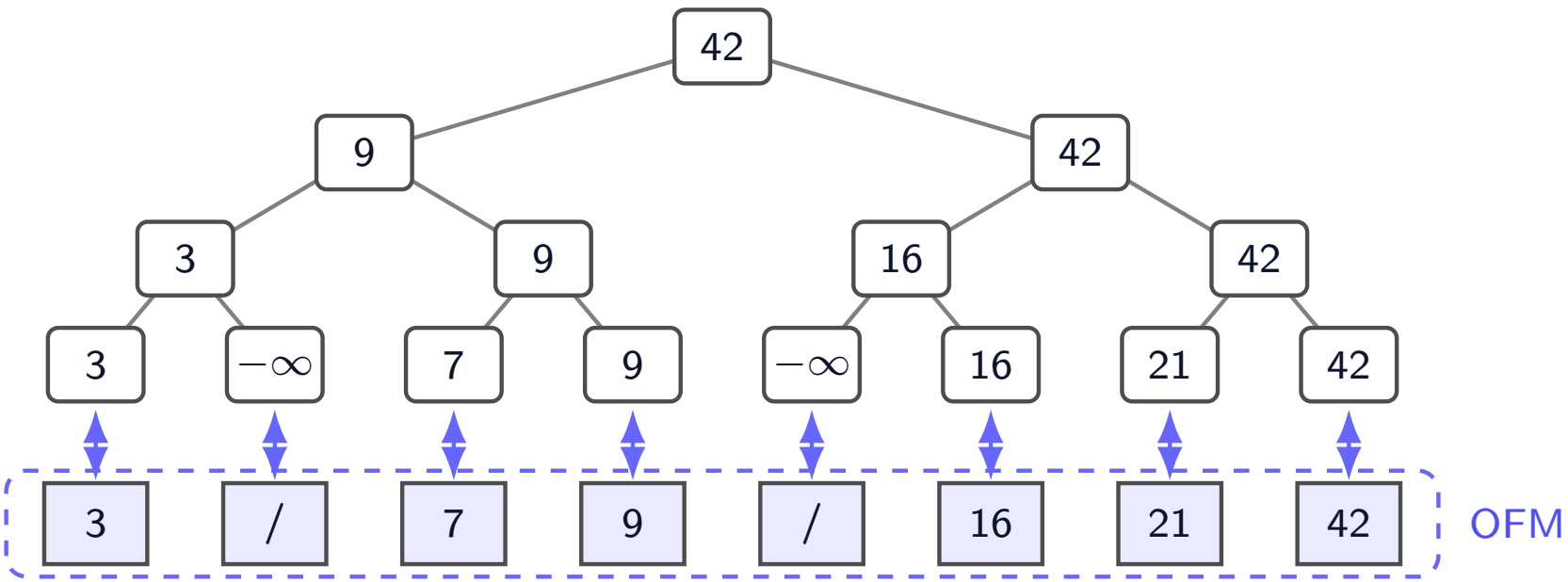
Ahora el hijo izquierdo tiene máximo $16 \geq 16$: izquierda.

Buscar



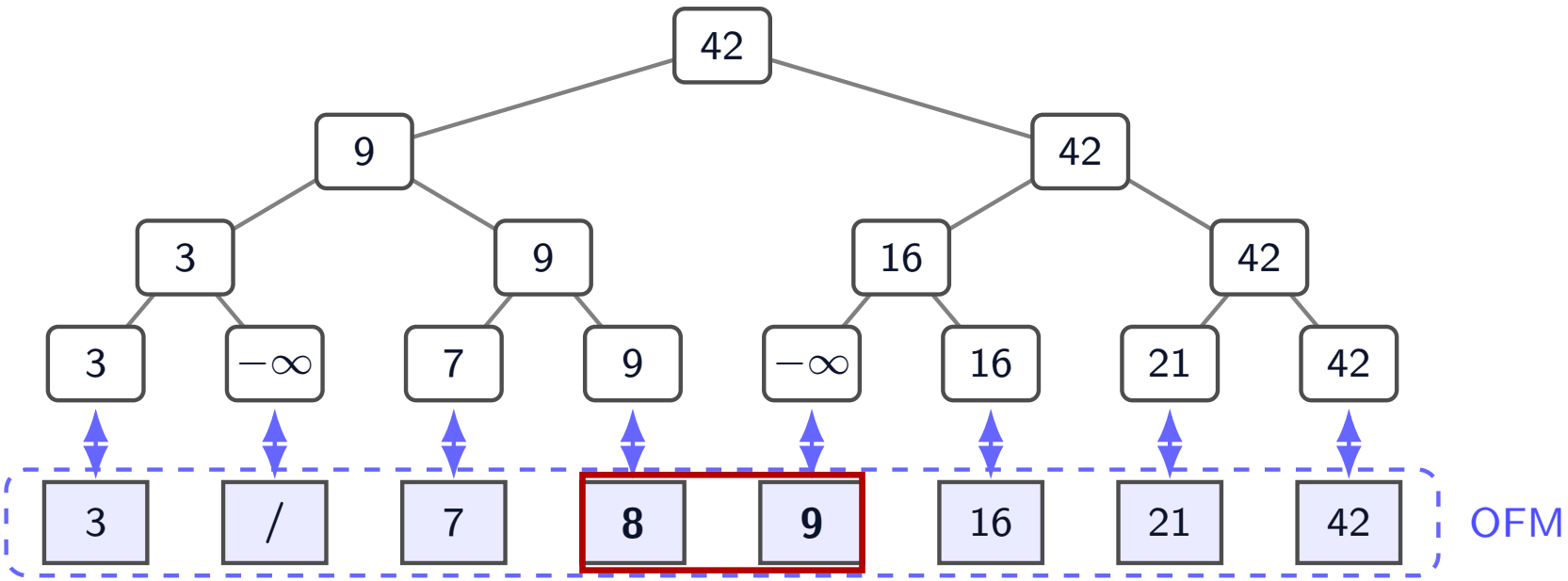
El hijo izquierdo es un hueco: derecha. Llegamos a la hoja 16, por un camino de la raíz a una hoja: $O(\log_{B+1} N)$ transferencias.

Paso 2: actualizar



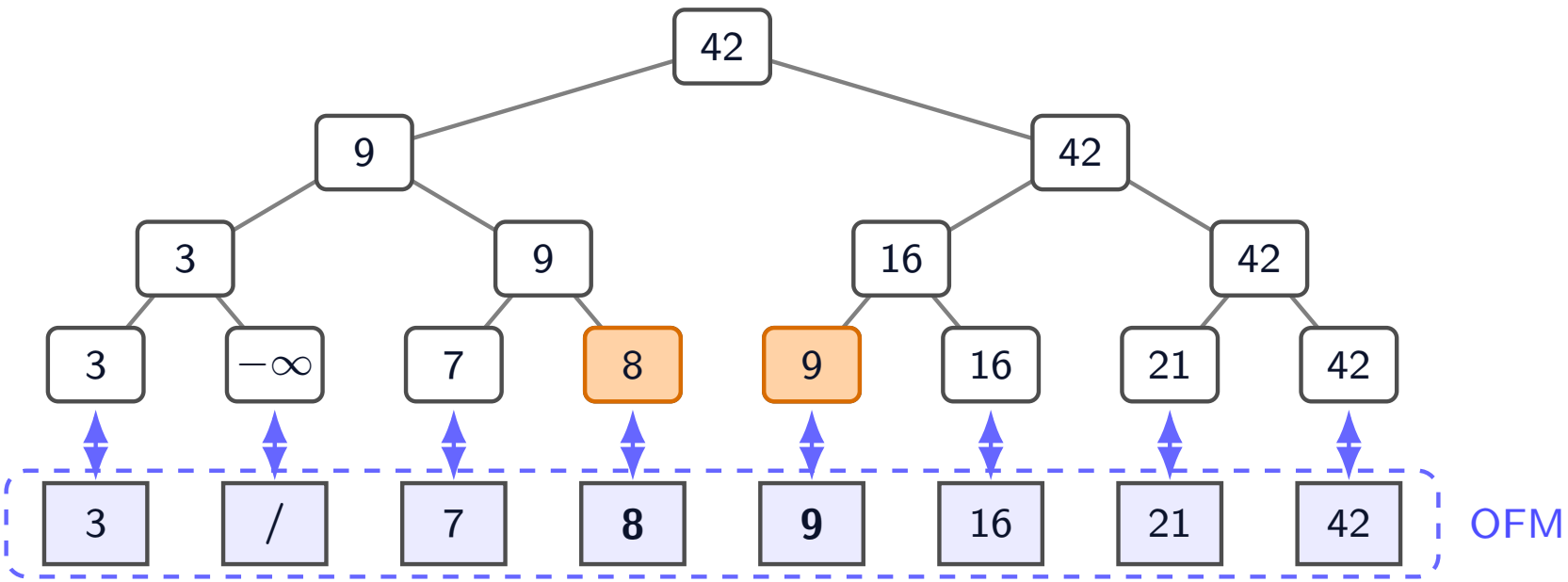
Insertar el 8. Primero se busca su sucesor, el 9: ya sabemos dónde va.

Paso 2: actualizar



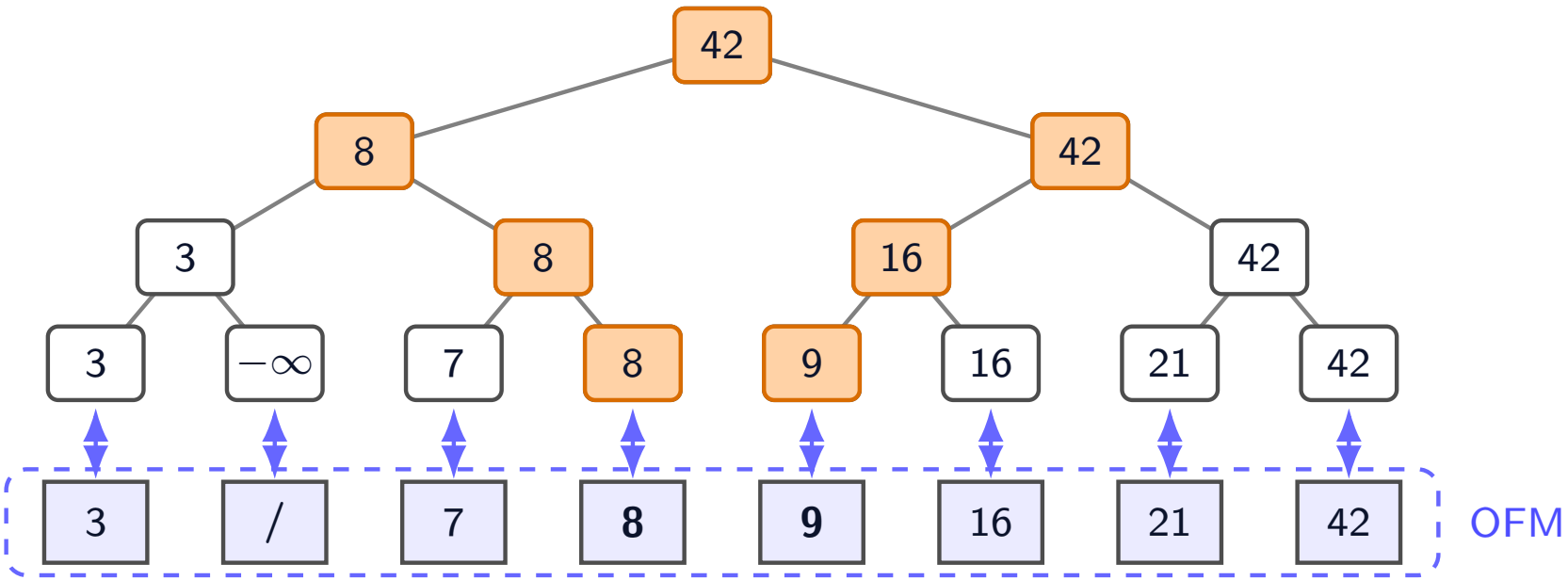
Se inserta en el OFM, que reacomoda un tramo de celdas consecutivas: el 9 se corre al hueco.

Paso 2: actualizar



Las hojas enlazadas con las celdas que cambiaron se actualizan.

Paso 2: actualizar



Y los máximos se recalculan hacia arriba, en **postorden**: cada nodo después de sus dos hijos.

¿Y la forma del árbol?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

No cambia

Una inserción cambia los **valores** del OFM y del árbol, pero no cuántas celdas hay ni qué hoja corresponde a cada celda.

¿Y la forma del árbol?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

No cambia

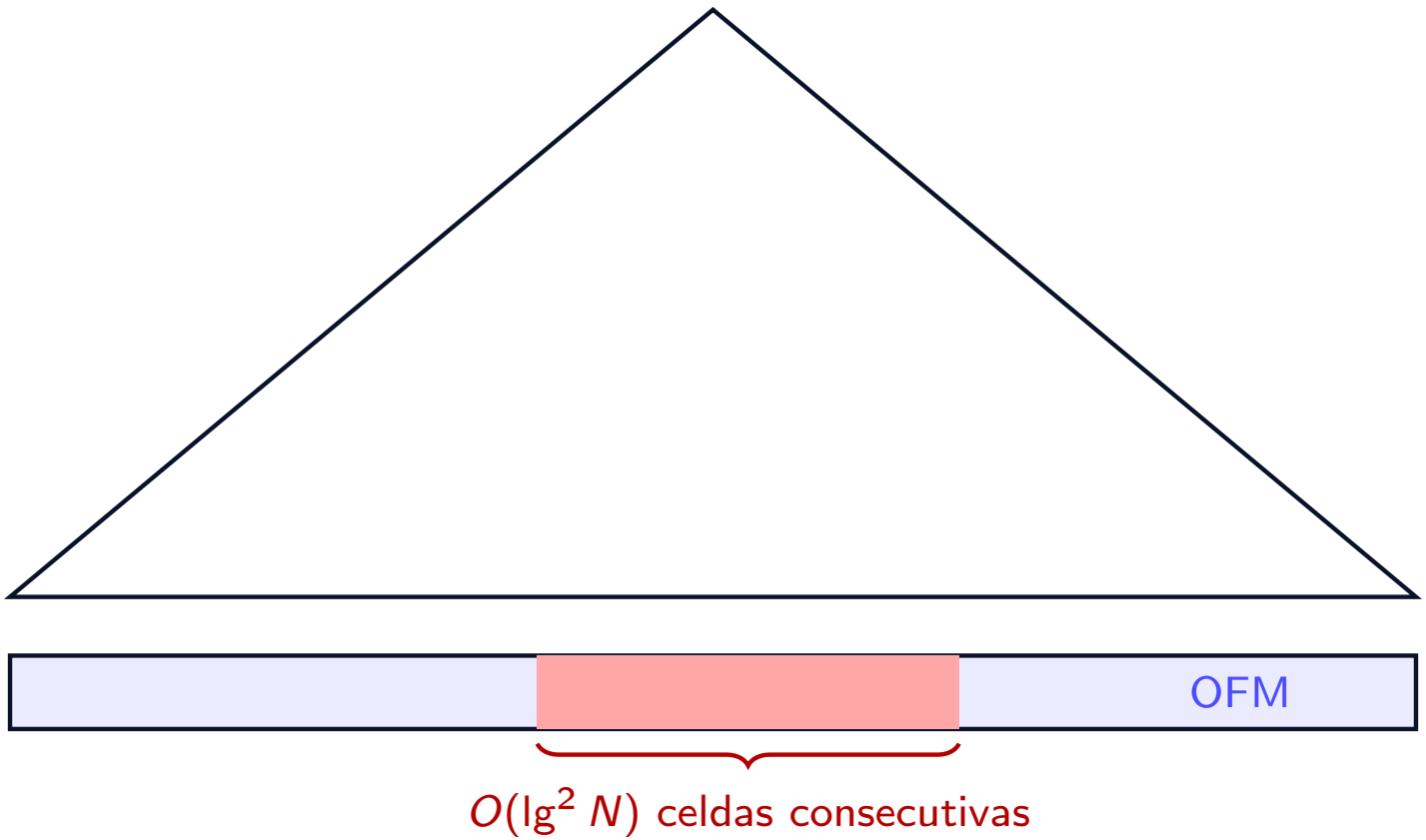
Una inserción cambia los **valores** del OFM y del árbol, pero no cuántas celdas hay ni qué hoja corresponde a cada celda.

Solo cuando el arreglo se queda chico o grande

Si la cantidad de elementos se duplica (o se reduce a la mitad), se reconstruye todo con un arreglo del tamaño adecuado. Como en una tabla dinámica, eso cuesta $O(1)$ amortizado por operación.

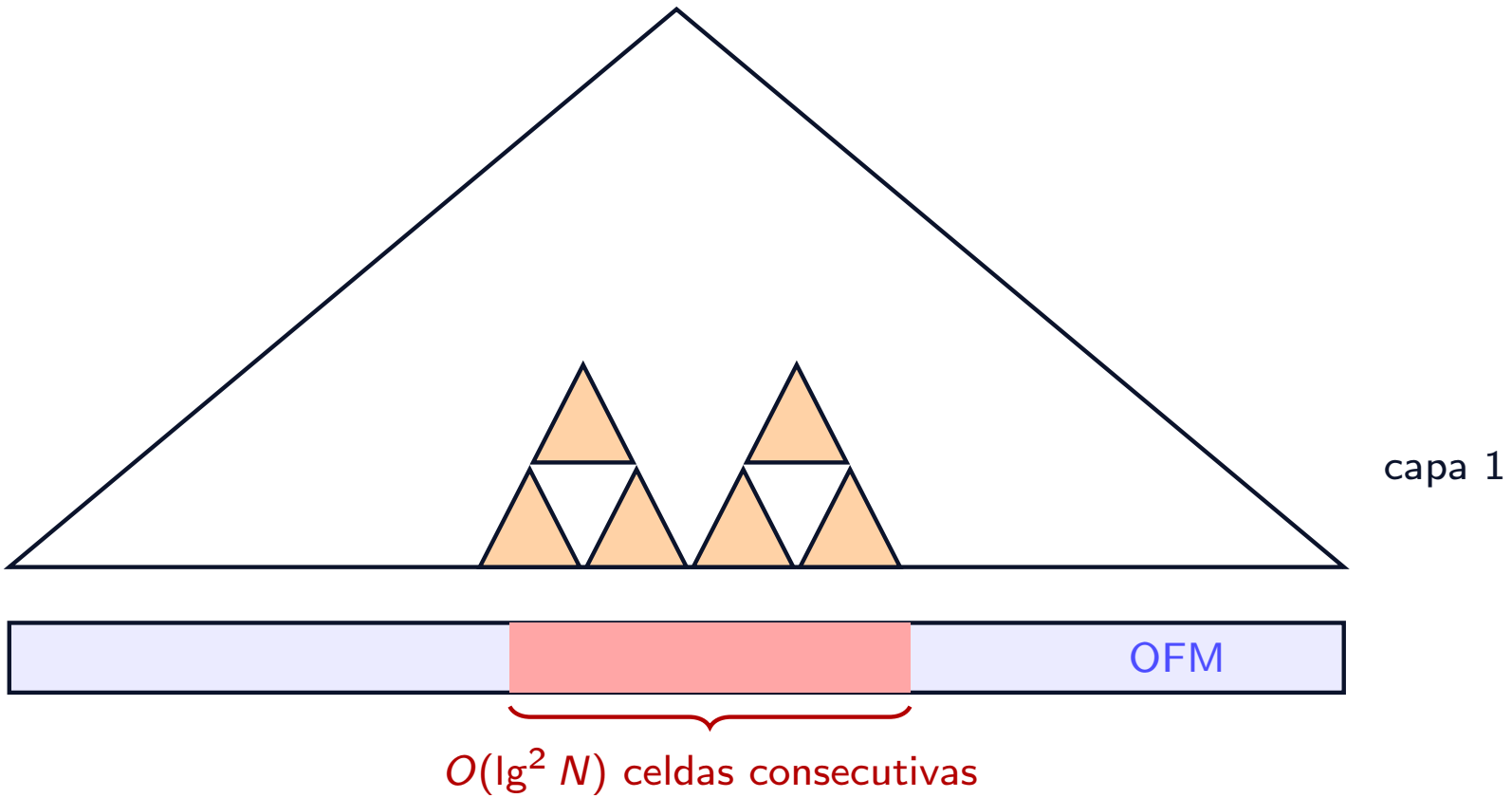
Estado actual: buscar cuesta $O(\log_{B+1} N)$. Falta saber cuánto cuesta **propagar** los cambios por el árbol.

Paso 3: ¿qué partes del árbol cambian?



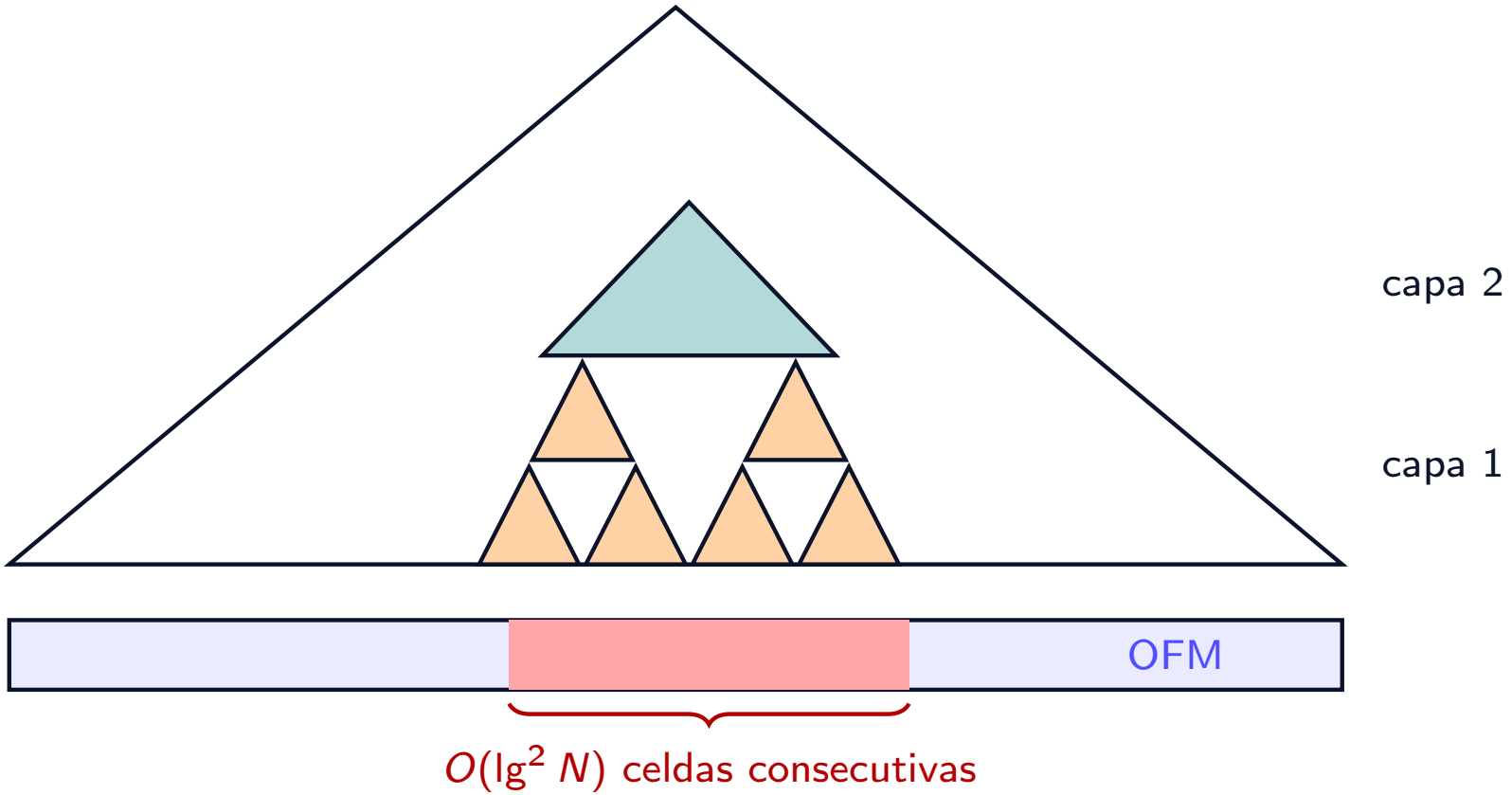
El OFM tocó un tramo de $O(\lg^2 N)$ celdas consecutivas. Hay que actualizar todos sus ancestros.

Paso 3: ¿qué partes del árbol cambian?



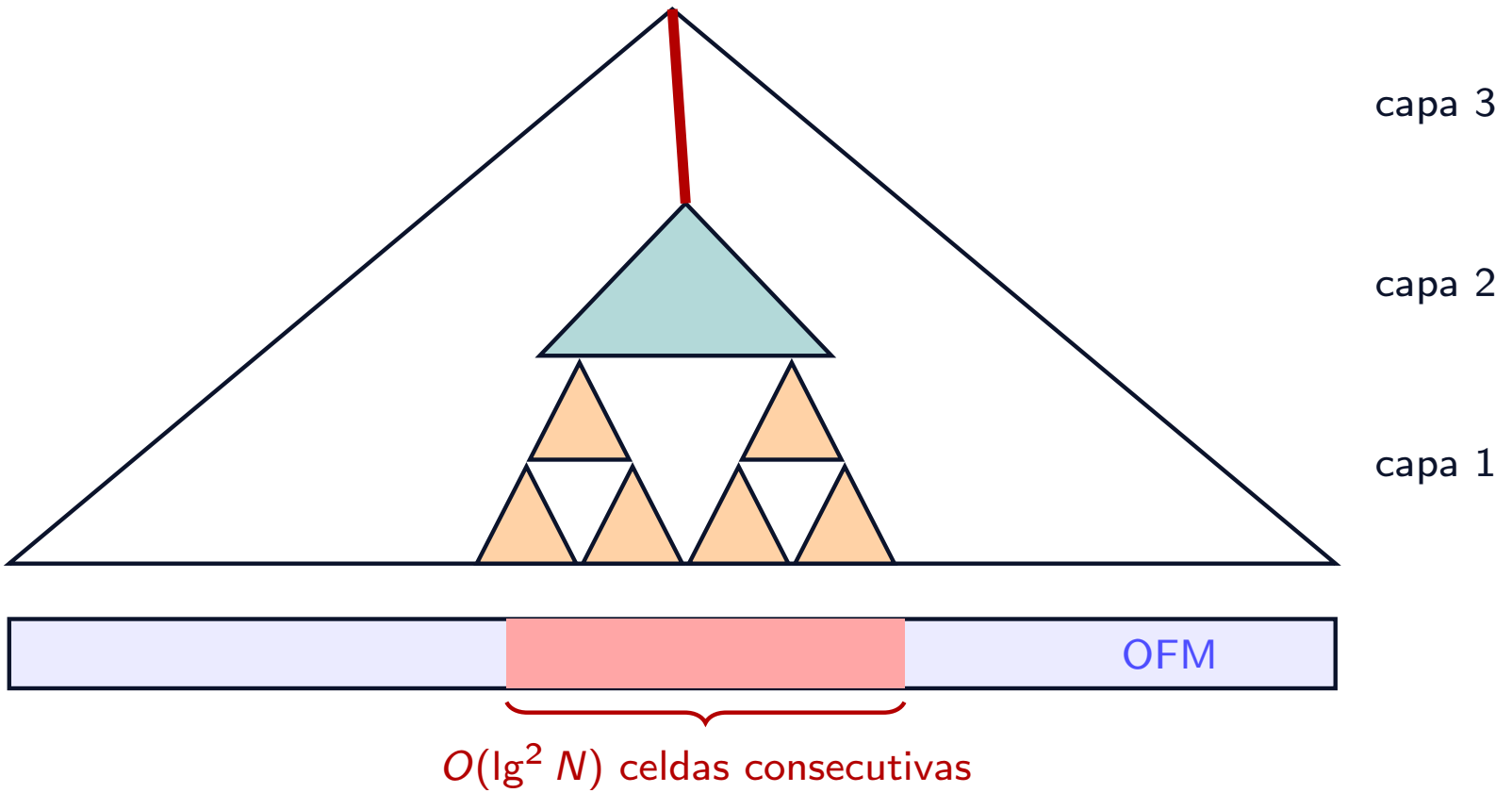
Capa 1: las piezas de tamaño $\leq B$ de los dos niveles más bajos de la recursión, sobre el tramo.

Paso 3: ¿qué partes del árbol cambian?



Capa 2: lo que queda entre esas piezas y el ancestro común más bajo de todo el tramo.

Paso 3: ¿qué partes del árbol cambian?



Capa 3: el camino desde ese ancestro común hasta la raíz.

Capa 1: las piezas pequeñas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Cada pieza cabe en dos bloques

Son piezas de a lo más B nodos consecutivos. Sobre un tramo de $O(\lg^2 N)$ hojas hay $O\left(1 + \frac{\lg^2 N}{B}\right)$ de ellas.

Capa 1: las piezas pequeñas

Cada pieza cabe en dos bloques

Son piezas de a lo más B nodos consecutivos. Sobre un tramo de $O(\lg^2 N)$ hojas hay $O\left(1 + \frac{\lg^2 N}{B}\right)$ de ellas.

El postorden las visita de forma ordenada

Al recorrer en postorden los dos niveles más bajos, el algoritmo alterna entre una pieza hija y su pieza padre. Con 4 bloques en memoria (dos por cada una) nunca hay que volver a traer una pieza ya visitada.

Costo de la capa 1: $O\left(1 + \frac{\lg^2 N}{B}\right)$ transferencias, lo mismo que costó modificar el OFM. Solo usamos que $M \geq 4B$.

Capas 2 y 3: hacia la raíz

Capa 2: hasta el ancestro común más bajo

Un árbol binario con $O\left(\frac{\lg^2 N}{B}\right)$ nodos. Podemos pagar una transferencia por **cada** nodo sin superar lo ya gastado.

Capas 2 y 3: hacia la raíz

Capa 2: hasta el ancestro común más bajo

Un árbol binario con $O\left(\frac{\lg^2 N}{B}\right)$ nodos. Podemos pagar una transferencia por **cada** nodo sin superar lo ya gastado.

Capa 3: del ancestro común a la raíz

Es un tramo de un camino hacia la raíz. Por el mismo argumento que en la búsqueda, cuesta $O(\log_{B+1} N)$.

Estado actual

Insertar y eliminar cuestan

$$O\left(\log_{B+1} N + \frac{\lg^2 N}{B}\right)$$

transferencias amortizadas.

¿Es suficiente?

Queremos $O(\log_{B+1} N)$, como el B-Tree

El término $\frac{\lg^2 N}{B}$ sobra. ¿Cuándo molesta?

¿Es suficiente?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Queremos $O(\log_{B+1} N)$, como el B-Tree

El término $\frac{\lg^2 N}{B}$ sobra. ¿Cuándo molesta?

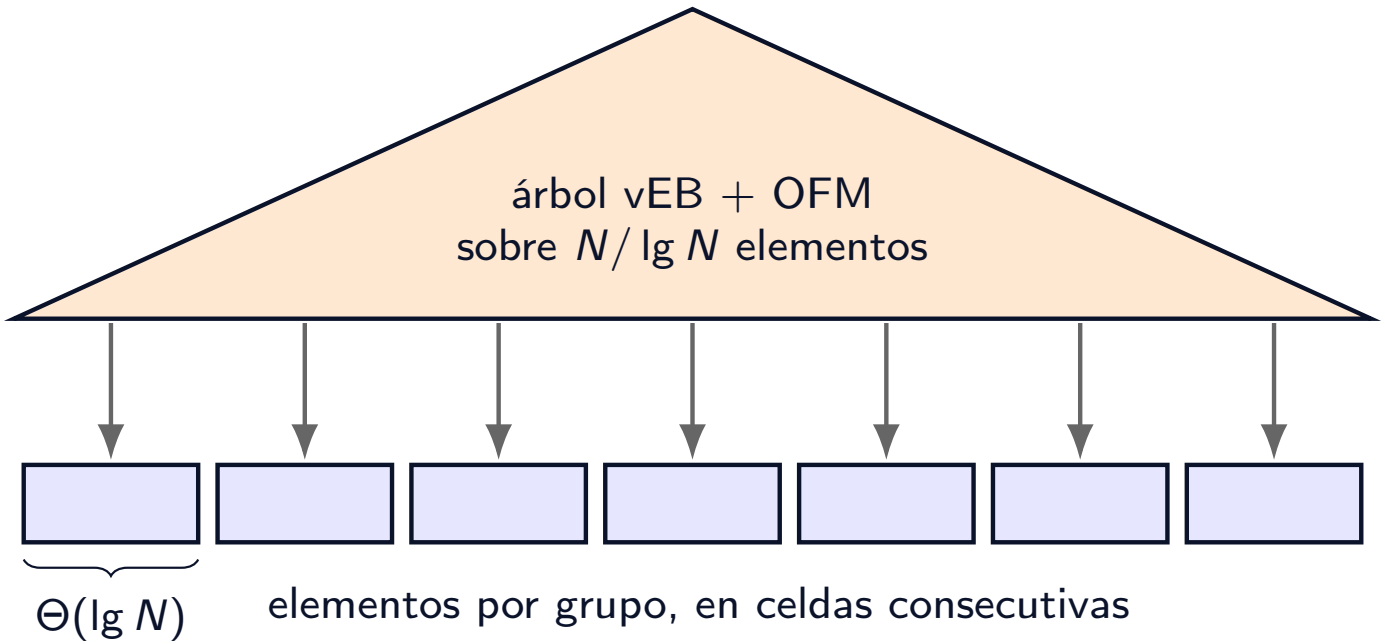
Comparando los dos términos

$\frac{\lg^2 N}{B} \leq \frac{\lg N}{\lg B}$ equivale a $B \geq \lg N \cdot \lg B$. Con bloques grandes no hay problema, pero si B es pequeño comparado con $\lg N$, el segundo término domina.

Como no conocemos B, la cota tiene que valer también para bloques pequeños. ¿Cómo se le quita un factor $\lg N$ al costo de actualizar?

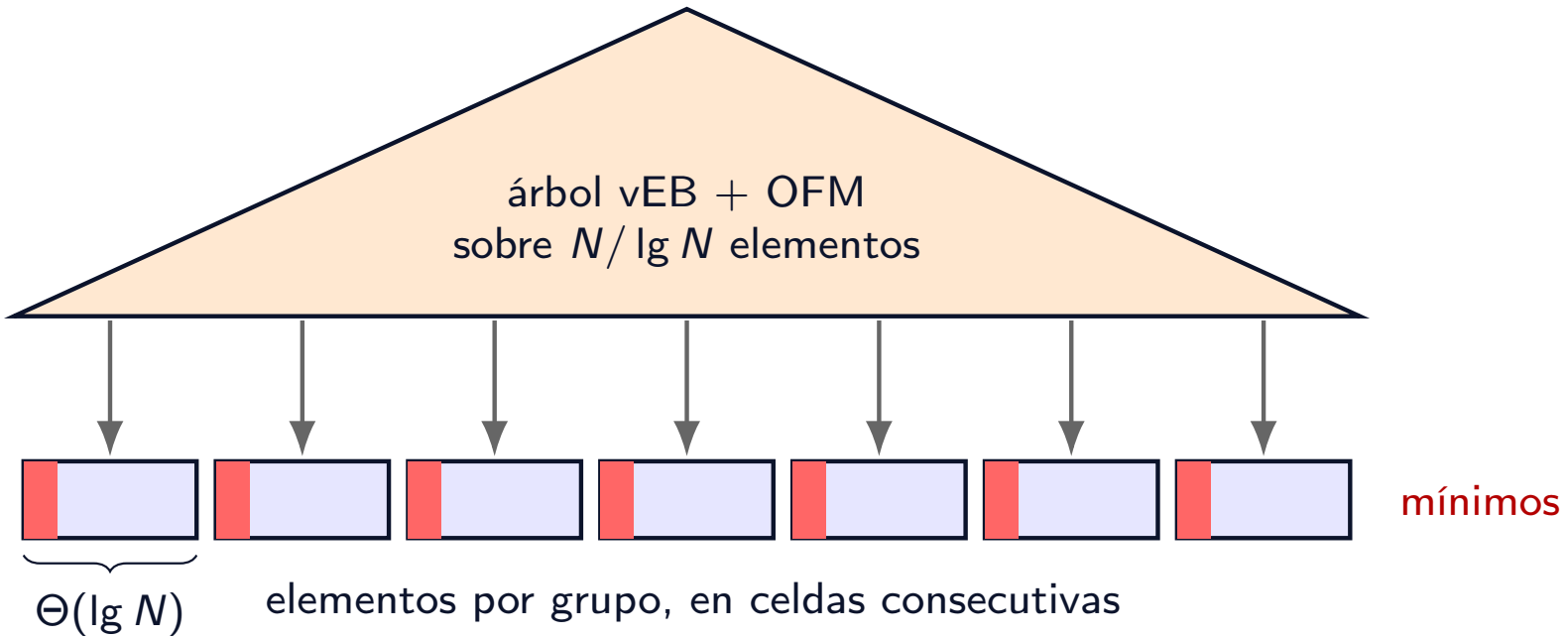
Haciendo que la estructura cara se actualice solo **una de cada $\lg N$ veces**.

Paso 4: indirección



Dos niveles

Paso 4: indirección



Dos niveles

Abajo, los N elementos en grupos de $\Theta(\lg N)$. Arriba, la estructura de los pasos 1 a 3, pero guardando solo **un representante por grupo**: su mínimo.

Buscar y actualizar con indirección

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Buscar

Arriba se ubica el grupo: $O(\log_{B+1} N)$. Abajo se recorre ese grupo: $O\left(1 + \frac{\lg N}{B}\right)$. Total $O(\log_{B+1} N)$.

Buscar y actualizar con indirección

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Buscar

Arriba se ubica el grupo: $O(\log_{B+1} N)$. Abajo se recorre ese grupo: $O\left(1 + \frac{\lg N}{B}\right)$. Total $O(\log_{B+1} N)$.

Insertar o eliminar

Se reescribe el grupo entero: $O\left(1+\frac{\lg N}{B}\right)$. La estructura de arriba no se entera, salvo que cambie el mínimo o el grupo cambie de tamaño.

Los grupos se mantienen entre un cuarto llenos y llenos

Un grupo que se llena se parte en dos; uno que queda muy vacío se fusiona con un vecino. Después de partir o fusionar, hacen falta $\Omega(\lg N)$ actualizaciones en ese grupo para que vuelva a ocurrir.

El costo amortizado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La estructura de arriba se toca una de cada $\Omega(\lg N)$ veces

Cada partición o fusión cuesta arriba $O\left(\log_{B+1} N + \frac{\lg^2 N}{B}\right)$, y se reparte entre las $\Omega(\lg N)$ actualizaciones que la provocaron.

El costo amortizado

La estructura de arriba se toca una de cada $\Omega(\lg N)$ veces

Cada partición o fusión cuesta arriba $O\left(\log_{B+1} N + \frac{\lg^2 N}{B}\right)$, y se reparte entre las $\Omega(\lg N)$ actualizaciones que la provocaron.

Sumando lo de arriba y lo de abajo

$$O\left(\frac{\log_{B+1} N + \frac{\lg^2 N}{B}}{\lg N} + \frac{\lg N}{B}\right) = O\left(\frac{\lg N}{B}\right)$$

El término molesto bajó de $\frac{\lg^2 N}{B}$ a $\frac{\lg N}{B}$, que es $O(\log_{B+1} N)$ para todo B. Y de todas formas hay que hacer una búsqueda para saber dónde insertar.

El resultado

Teorema (cache-oblivious B-Tree)

Existe una estructura que no conoce B ni M y soporta SEARCH, INSERT y DELETE en $O(\log_{B+1} N)$ transferencias (amortizado para las actualizaciones).

El resultado

Teorema (cache-oblivious B-Tree)

Existe una estructura que no conoce B ni M y soporta SEARCH, INSERT y DELETE en $O(\log_{B+1} N)$ transferencias (amortizado para las actualizaciones).

Ingrediente	Para qué sirve
Layout de van Emde Boas	Buscar en $O(\log_{B+1} N)$ sin conocer B
Ordered File Maintenance	Insertar manteniendo el orden físico
Máximos y postorden	Propagar cambios con localidad
Indirección	Quitar un factor $\lg N$ a las actualizaciones

Queda una deuda

Todo se apoya en que el OFM reacomoda solo $O(\lg^2 N)$ celdas consecutivas por actualización. La próxima clase abrimos esa caja.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Layout de van Emde Boas: cortar el árbol a media altura, guardar la parte superior y luego las inferiores, recursivamente.
- En el nivel de la recursión con piezas de tamaño entre $\sqrt{B}$ y B, un camino cruza $\leq 2 \log_B N$ piezas de ≤ 2 bloques: buscar cuesta $O(\log_{B+1} N)$.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Layout de van Emde Boas: cortar el árbol a media altura, guardar la parte superior y luego las inferiores, recursivamente.
- En el nivel de la recursión con piezas de tamaño entre $\sqrt{B}$ y B, un camino cruza $\leq 2 \log_B N$ piezas de ≤ 2 bloques: buscar cuesta $O(\log_{B+1} N)$.
- Versión dinámica: las claves en un OFM, y encima un árbol de máximos con ese layout. Actualizar cuesta $O\big(\log_{B+1} N + \frac{\lg^2 N}{B}\big)$.
- Con indirección (grupos de $\Theta(\lg N)$ elementos), las actualizaciones bajan a $O(\log_{B+1} N)$.

Logros de la sesión



Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos escribir el layout de van Emde Boas de un árbol pequeño

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos escribir el layout de van Emde Boas de un árbol pequeño
- Sabemos demostrar la cota $4 \log_B N$ de la búsqueda

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos escribir el layout de van Emde Boas de un árbol pequeño
- Sabemos demostrar la cota $4 \log_B N$ de la búsqueda
- Sabemos describir cómo se combinan el árbol y el OFM, y analizar el costo de propagar una actualización

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos escribir el layout de van Emde Boas de un árbol pequeño
- Sabemos demostrar la cota $4 \log_B N$ de la búsqueda
- Sabemos describir cómo se combinan el árbol y el OFM, y analizar el costo de propagar una actualización
- Sabemos explicar qué resuelve la indirección y por qué el resultado final es $O(\log_{B+1} N)$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

Bibliografía citada

- Prokop, H. (1999). *Cache-oblivious algorithms*. Tesis de maestría, MIT.
- Bender, M. A., Demaine, E. D., & Farach-Colton, M. (2000). Cache-oblivious B-trees. *Proceedings of FOCS'00*, 399–409.
- Bender, M. A., Duan, Z., Iacono, J., & Wu, J. (2002). A locality-preserving cache-oblivious dynamic dictionary. *Proceedings of SODA'02*, 29–38.
- Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. `https://courses.csail.mit.edu/6.851/spring21/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.